Atlas - System Design Document

Version: 0.2.0 (TARGET CONTRACTS, NOT VERIFIED RUNNING SYSTEM)
Date: 2026-10-14 | Repository: Swayamjain1240/Atlas | Related: Architecture, Requirements, Master

1. Design principles

- One public API owner: Express. Internal Python FastAPI supports AI only and is loopback-only with mutual authentication via private secret.
- Single-writer persistence: Node owns SQLite writes; Python owns ChromaDB writes. No shared writes or hidden cross-store transactions.
- Idempotent ingestion, explicit source consent, durable jobs, traceable citations and safe local file access.
- Standard JSON responses and stable versioned API contracts; log task IDs rather than sensitive document contents.

2. Runtime service definitions

2.1 Frontend

- React component tree: App shell, Header, Sidebar, Chat, Dashboard, Timeline, Settings and Three.js KnowledgeGraph.
- Vite handles local dev proxy; production frontend communicates with Express via same-origin /api/v1.
- GSAP animations must be initialized/cleaned up in component lifecycle; respect prefers-reduced-motion.
- API client centralizes Authorization headers, retry rules, timeouts, errors and cancellation.

2.2 Express

- app.js assembles security middleware, routes and error handling; server.js validates settings and binds loopback.
- Route -> validator -> controller -> service -> SQLite or Python client. No database operations in controllers.
- Module groups: query, ingestion, memory, history, stats, settings, health and private AI bridge.

2.3 Python AI worker

- B.2 FastAPI internal endpoints: authenticated GET /internal/health, POST /internal/embed with {texts:[string]}, and POST /internal/generate with {prompt:string,system?:string}.
- B.2 modules: validated worker configuration, internal bearer authentication, Sentence-Transformers embedding adapter and bounded Ollama adapter.
- Later B.3-B.5 endpoints add ingestion, search, retrieval and persistence only after their consent/data contracts are implemented.
- Internal API always authenticates; never accept arbitrary local paths from untrusted callers without allowlist/consent.

3. SQLite schema (proposed)

3.1 sources

- source_id TEXT PRIMARY KEY; type TEXT NOT NULL; root_path TEXT NOT NULL; consent_at TEXT; enabled INTEGER; scan_interval_minutes INTEGER; created_at TEXT; updated_at TEXT.
- Unique (type, normalized_root_path). Browser history sources store browser/profile identifier, not scraped website content.

3.2 memories

- memory_id TEXT PRIMARY KEY; source_id TEXT REFERENCES sources; source_path TEXT; title TEXT; content_hash TEXT; text_preview TEXT; size_bytes INTEGER; created_at TEXT; ingested_at TEXT; accessed_count INTEGER DEFAULT 0; last_accessed TEXT; metadata_json TEXT.
- Index (source_id, ingested_at), (content_hash), (source_path).

3.3 ingestion_jobs

- job_id TEXT PRIMARY KEY; source_id TEXT; state TEXT CHECK queued/running/completed/partial/failed/canceled;

discovered_count INTEGER; indexed_count INTEGER; skipped_count INTEGER; failed_count INTEGER; error_code TEXT; created_at TEXT; updated_at TEXT.

3.4 chat_sessions and chat_messages

- chat_sessions(session_id TEXT PRIMARY KEY, created_at TEXT, updated_at TEXT, title TEXT).
- chat_messages(message_id TEXT PRIMARY KEY, session_id TEXT REFERENCES chat_sessions, role TEXT, content TEXT, sources_json TEXT, created_at TEXT).

3.5 app_settings and schema_migrations

- app_settings(key TEXT PRIMARY KEY, value_json TEXT, updated_at TEXT).
- schema_migrations(version INTEGER PRIMARY KEY, applied_at TEXT).
- Enforce foreign keys; enable WAL with controlled writes; regular backups with SQLite online backup API.

4. ChromaDB schema and IDs

- Collection atlas_memory; embedder: all-MiniLM-L6-v2 producing 384-dimensional vectors; cosine distance index.
- One record per chunk: chunk_id = SHA256(source_id + normalized_path + content_hash + chunk_index + embedding_model_version), truncated only if collision-resistance remains sufficient (recommend full hash).
- Each record: document=chunk text, embedding=float[384], metadata={memory_id,source_id,source_type,source_path,t itle,timestamp,chunk_index,content_hash}.
- Chroma metadata values must be supported scalar types; JSON-encode tags or normalize elsewhere. Never store arbitrary arrays as metadata.
- Changing embedding model or dimensions requires a new collection/version and re-embedding. Track migration and rollback.

5. Ingestion state machine

- CREATED -> QUEUED -> RUNNING -> COMPLETED / PARTIAL / FAILED / CANCELED.
- Discover only approved roots; canonicalize paths; reject symlink escapes and hidden credential folders.
- Parse .txt,.md,.py,.js,.json,.csv,.html,.css,.yaml/.yml; plus .pdf and .docx.
- Initial chunking contract: 1500 characters with 200-character overlap; preserve source/page/line where available; content hash avoids duplicates.
- Apply bounded queue, file size/count limits and CPU worker concurrency; log failures without aborting unrelated documents.
- Upsert chunks and metadata with compensating actions: orphan-vector sweep and failed-state reconciliation on restart.
- Browser history: Chrome, Edge and Brave; copy SQLite history database to a temporary snapshot when locked, parse permitted visit fields, respect consent and OS profile protection. Browsing content is not inferred from URLs.

6. Query and citation pipeline

- Input: query string (1-2000 chars); optional session_id, source filter, top_k (1-10), conversation_context toggle.
- Retrieval: embed query, cosine search with source filters, score normalization, optional dedup/rerank later.
- Context: cap token budget, include clear source boundaries, defend against prompt-injection in retrieved text.
- Ollama: local llama3.2:3b via configured loopback URL; temperature default 0.1; timeout and cancellation supported.
- Generation: answer grounded in retrieved chunks; clearly state insufficient evidence; never invent file citations.
- Response sources include chunk_id, memory_id, title, source_type, source_path, optional page/line, relevance score.
- On Ollama failure, use a clearly labeled retrieval-only result; do not present it as an AI-generated answer.

7. Public API contract (Express)

- GET /api/v1/health: public liveness (safe data only).
- GET /api/v1/status: authenticated component status (frontend uses this currently).
- POST /api/v1/query: authenticated grounded chat; payload {query, session_id?, source_type?, top_k?}.

- POST /api/v1/ingest: authenticated job creation; payload {source_id?, sources?}; resolve sources against saved consent/allowlist.
- GET /api/v1/ingest/jobs/:jobId: authenticated job status.
- GET /api/v1/stats: authenticated memory counts, sources, recent jobs, model readiness.
- GET /api/v1/memory/recent?limit=&cursor=: authenticated paginated timeline.
- DELETE /api/v1/memory/:id: authenticated memory deletion and linked chunk cleanup.
- POST /api/v1/search: authenticated similarity search without LLM generation.
- GET/POST /api/v1/settings: authenticated approved settings only; no API secret echoing.
- GET /api/v1/chat/sessions and GET /api/v1/chat/sessions/:id/messages: authenticated history.
- All routes except public health must enforce auth, validation, rate limiting, safe errors and data permissions.

8. Standard request and response

- Success JSON: {success:true,data:{...},meta:{request_id,processing_ti me_ms,version:"0.2.0"}}.
- Failure JSON: {success:false,error:{code:"VALIDATION_ERROR",message:" Safe explanation"},meta:{request_id}}.
- Expected errors: UNAUTHORIZED, FORBIDDEN, VALIDATION_ERROR, NOT_FOUND, RATE_LIMITED, SOURCE_NOT_ALLOWED, INGESTION_FAILED, MODEL_UNAVAILABLE, TIMEOUT, INTERNAL_ERROR.
- Never include Python stack traces, file content, keys or full local absolute paths in remote-facing errors.

9. Application security and trust model

- API key validated with constant-time equality; rotate on compromise. Optional short-lived session token for frontend storage.
- Express: Helmet/CSP with no unsafe-inline in production, strict local CORS, IP+route rate limits, input schemas (Zod/Joi), body caps and timeouts.
- Localhost is not itself an authorization boundary: browser-based attacks can access local services. Require a real credential and strict Origin checks.
- Python worker requires internal secret and binds 127.0.0.1; disable docs and debug externally.
- Constrain file paths using realpath and explicit directory grants; prevent SSRF if URL ingestion is ever added.
- Logs redact credentials and document bodies; encrypted disk recommended; no cloud telemetry. .env and data/ are ignored.
- Dependabot/npm audit/pip-audit/Bandit, automated secret scan and migration rollback are release gates.

10. UI behavior and one-viewport contract

- Single viewport for authenticated shell; do not create page-level scrolling. Long chat/timeline may scroll inside bounded panels; paginated data lists.
- Dashboard uses real /stats values and distinguishes offline/unknown, never hardcodes "Online".
- Settings controls approved directories, model selection, job schedules and credential lifecycle; errors displayed with recovery guidance.
- Three.js knowledge graph maps to actual indexed memories, not random visual nodes; lazy load and WebGL fallback required.
- Reduced motion, focus indicators, keyboard navigation and mobile-responsive layouts are acceptance criteria.

11. Performance and observability

- Preserve targets from v0.1: p95 complete query under 2s, vector search under 200ms for up to 100K documents, idle RAM under 500MB, typical Ollama generation under 5s, ingestion 100 text files/s.
- These are unverified design targets, not promises; Llama 3.2 on CPU may exceed them. Benchmark by hardware/model/data size, measure p50/p95 separately for search and generation.
- Instrument timing for extraction, chunking, embedding, retrieval, generation, SQLite and API gateway; no PII-bearing telemetry.
- Apply query cache only when indexed state/model permissions match; invalidate on ingest/delete and redact stored

prompts where configured.

12. Tests and acceptance criteria

- Backend: Vitest + Supertest for public/private endpoints, auth, validation, CORS, rate limits and SQLite migrations.
- Frontend: Vitest + React Testing Library; accessibility; UI error, loading, offline and WebGL fallback.
- AI: Pytest + httpx; deterministic mocked Ollama; embedding/chunk/filter, Chroma CRUD and prompt-injection resistance.
- Integration: ingestion -> embeddings -> Chroma -> Express stats -> query -> citations -> deletion + history.
- E2E: Playwright on Windows, plus macOS/Linux smoke; verify fresh offline run after dependencies/models cached.
- Security: npm audit, pip-audit, Bandit, secret scanning, malicious file paths, oversized files, CSRF/local-origin abuse.
- Load: Locust or equivalent; record p95 results, hardware profile and model version.
- No migration stage is complete until its acceptance tests and manual verification pass.